Homework — 1.2.2 Applications Generation — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [3] (a) Utility — it maintains/optimises the computer's files/storage (1). (b) Application — it lets the user perform a real-world task (1). (c) Utility — it protects/maintains the system rather than doing user work (1).

2. [3] Open source advantages (1 each, max 2): free / no licence cost; source code can be viewed and modified to suit the school; large community support; not tied to one vendor. Closed source advantage (1): official/dedicated vendor support and warranties; often more polished / fully tested / regular professional updates. Max 3.

3. [3] Lexical analysis converts the source code into a series of **tokens** (1); whitespace and comments are removed (1). The symbol table stores identifiers (variables, subroutines) and their attributes so later stages (syntax analysis, code generation) can look them up consistently / check usage (1).

4. [2] Code optimisation refines/rearranges the generated code so it runs faster and/or uses less memory while producing the same result (1). It is included because the basic generated code may be inefficient; optimisation improves performance of the final program, which matters for software run many times (1).

5. [2] The loader is OS software that copies the executable from secondary storage into RAM (1) and prepares it to run (e.g. sets up addresses / hands control to the program) (1).

6. [1] With dynamic linking the library (e.g. a .dll) is a separate file linked at run time, so replacing/updating that one shared file updates every program that uses it, without re-linking or re-releasing the executable (1).

Part B (6 marks)

7. [3] Any three (1 each): CPU checks the interrupt register at the end of the current FDE cycle; finishes the current instruction / compares priorities; pushes the registers (PC, ACC) onto the stack; loads the address of and runs the correct ISR; pops the registers off the stack to resume. The stack is LIFO so nested/higher-priority interrupts resume the earlier task in the correct order. Max 3.

8. [1] Any one (1): RISC has a small/simple instruction set with fixed-length instructions executed in (typically) one cycle, whereas CISC has many complex variable-length instructions that may take several cycles; RISC relies more on software/compiler and pipelining; CISC does more per instruction. Max 1.

9. [2] Von Neumann uses a **single shared memory and bus** for both instructions and data (1), whereas Harvard uses separate memories/buses for instructions and data (1).

Total: 14 + 6 = 20